
Generalization in DQN with Actor

ActorDQN vs DQN

Xiaojie Gu

University of Wuerzburg

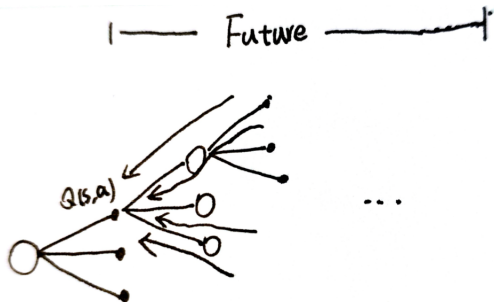
First Advisor: Mr. Sebastian Griesbach | Supervisor: Prof. Dr. Carlo D'Eramo

This presentation was made with assistance from GPT-6 Sol under the guidance of my hand draft.

Outline

1. Introduction
2. Environment design
3. Model Design
4. Experimental results

Introduction: generalization in DQN



Mapping directly

S

O

• a1

• a2

• a3

Introduction: Another viewpoint

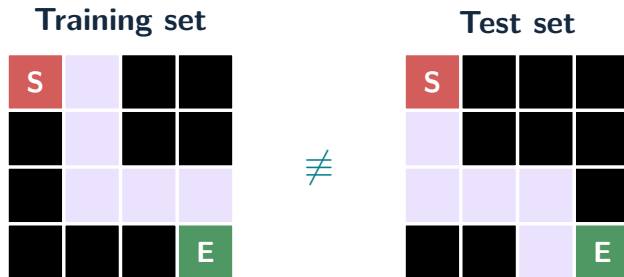
Hypothesis: Classification is easier. It might use a general pattern that leads to better generalization.

	Book 1	Book 2	Book 3	Book 4	
Regression	0.7	0.4	0.23	0.92	label
	0.1	0.4	0.1	0.6	output
Classification	×	×	×	✓	learn the best-book label directly

Environment Design

1. Regulation
2. Procedural maze

Regulation: no shared optimal solution



$$\mathcal{P}_{\text{train}} \cap \mathcal{P}_{\text{test}} = \emptyset$$

Each original maze and its variations stay in one split; the intended shortest path does not cross splits.

Procedural Maze

1. Enumerate start-goal pairs

S	E	1
2	3	4
5	6	7

S	1	E
2	3	4
5	6	7

...

S	1	2
3	4	5
6	7	E

1	S	E
2	3	4
5	6	7

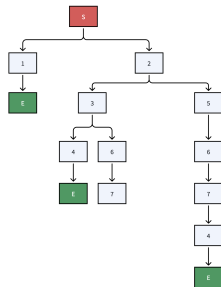
1	S	2
E	3	4
5	6	7

...

1	S	2
3	4	5
6	7	E

2. Generate valid paths

S	1	E
2	3	4
5	6	7



S	1	E
2	3	4
5	6	7

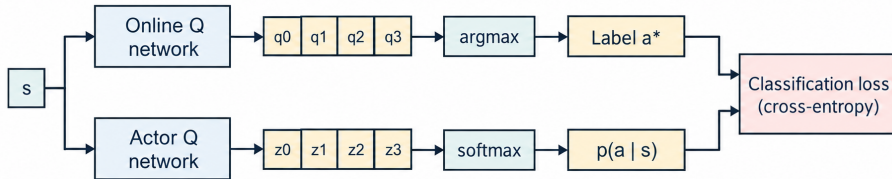
S	1	E
2	3	4
5	6	7

S	1	E
2	3	4
5	6	7

Model Design

1. ActorDQN

Model Design: ActorDQN



Model Design: ActorDQN

```
91     def update_qnetwork(self,):
92         # a list of (s,a,r,s')
93         batch = self.replaybuffer.sample(batch_size=self.sample_batch_size)
94
95         #  $r + \gamma \max_{a'} Q(s', a')$ 
96         td_target = self._td_target(batch)
97
98         #  $Q(s, a)$ 
99         prediction = self.qnetwork(batch.states)
100         estimation = prediction.gather(1, batch.actions)
101
102         #  $(Q_{sa} - TD\_target)^2$ 
103         loss = self.loss_fn(td_target, estimation)
104         self.optim.zero_grad()
105         loss.backward()
106         self.optim.step()
107
108         # update for actor qnetwork
109         label = prediction.argmax(dim=1)
110         logits = self.actor_dqn_network(batch.states)
111         loss_actor = self.loss_fn_actor(logits, label)
112         self.optim_actor.zero_grad()
113         loss_actor.backward()
114         self.optim_actor.step()
```

Experimental Results

1. Performance in mazes

Full held-out sets for 4×4 and 5×5 mazes.

2. Justification for the idea

Performance in Maze: 4×4

Training set: 803 | Evaluation set: 201
Step limit: 160

Policy	Success rate	Mean step	Optimal length
ActorDQN	0.828 ± 0.029	49.01 ± 6.60	6.163 ± 0.143
Basic DQN	0.111 ± 0.072	142.76 ± 11.22	6.163 ± 0.143
Uniform Random	0.740 ± 0.048	88.87 ± 5.74	6.163 ± 0.143

Performance in Maze: 5×5

Training set: 11,891 | Evaluation set: 2,973
Step limit: 250

Policy	Success rate	Mean step	Optimal length
ActorDQN	0.673 ± 0.027	114.11 ± 8.80	10.236 ± 0.014
Basic DQN	0.129 ± 0.036	218.71 ± 8.52	10.236 ± 0.014
Uniform Random	0.564 ± 0.004	178.01 ± 0.43	10.236 ± 0.014

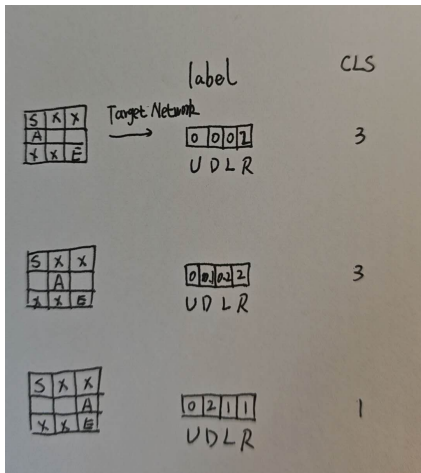
Generalization Gap

Gap = training-set mean – held-out mean;

Maze	Policy	Δ success rate	Δ mean step
4×4	ActorDQN	+0.026	−6.07
	Basic DQN	+0.102	−15.72
5×5	ActorDQN	+0.014	−3.89
	Basic DQN	+0.008	−1.83

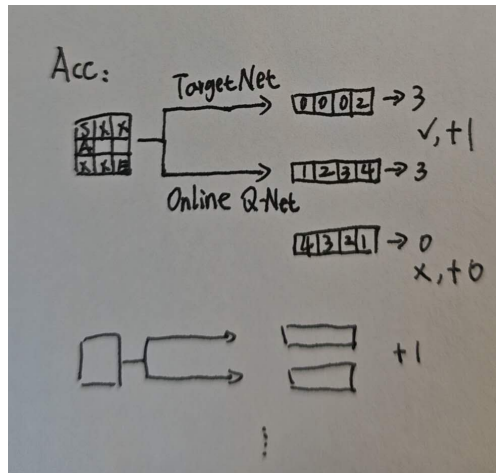
A smaller absolute gap is better.

Justification for the Idea



Classification vs. Q-value Regression

Target Network coming from	Task Type	here ↓ Train Eval □ □	
		Acc	Success Rate
DQN	CLS	0.899	137/201
	Reg	0.853	116/201



Thanks